Universidad Nacional de Tucumán

Facultad de Ciencias Exactas y Tecnología



Simulación y paralelización de caminatas aleatorias (Random Walk) con MPI

Proyecto 5 · Caminata del Borracho

Casos principales: 1D y 2D | Extensión: 3D y 4D

Integrantes

Rodriguez Bernal Javier Alejandro · Rollan Natalia · Salcedo Cristian Facundo

Métodos Numéricos II

Departamento de Ciencias de Computación

2026

Resumen

Se desarrolló en C, mediante MPI, un simulador de caminatas aleatorias para los casos 1D y 2D solicitados en la consigna. Como ampliación del trabajo, se implementó una versión parametrizable en D dimensiones y se ejecutaron también pruebas en 3D y 4D. Las caminatas completas se distribuyen entre procesos independientes, con generación pseudoaleatoria PCG32 y una reducción colectiva de resultados al finalizar.

La validación principal comparó la distancia promedio al origen con las expresiones teóricas y examinó histogramas de posiciones finales. Para medir el rendimiento se utilizó un problema fijo de diez millones de caminatas y 1000 pasos, con cinco ejecuciones por configuración y tiempos resumidos mediante la mediana. Se evaluaron cuatro equipos: un clúster limitado a una cuota de 12 CPU equivalentes y tres computadoras personales.

En el clúster, con 12 procesos, el speedup fue 11,978 en 1D y 11,996 en 2D, con eficiencias de 99,82 % y 99,97 %, respectivamente. En los equipos personales aparecieron diferencias asociadas al número de núcleos físicos, los hilos lógicos y las condiciones de ejecución. Se detectó también que la implementación especializada de 2D tiene un costo por paso notablemente superior al de la versión general; se propone investigarlo mediante perfiles y pruebas controladas.

> Alcance: 1D y 2D constituyen el núcleo de la evaluación. 3D y 4D se presentan de forma sintética como extensión funcional y experimental del equipo.

Índice

1. Introducción 3
1.1 Fundamento teórico 3
1.2 Objetivos 3
2. Datos, implementación y metodología 4
2.1 Configuración de los experimentos 4
2.2 Estrategia de paralelización 4
2.3 Implementación MPI y generación aleatoria 4
2.4 Medición y reproducibilidad 5
3. Resultados 5
3.1 Validación de 1D y 2D 5
3.2 Distribución de posiciones finales 6
3.3 Escalabilidad en el clúster 7
3.4 Comparación entre los cuatro sistemas 8
3.5 Tiempo absoluto y anomalía de 2D 9
3.6 PC de dos núcleos: hilos lógicos 10
3.7 Extensión a 3D y 4D 10
4. Conclusiones 10
4.1 Limitaciones 11
4.2 Mejoras recomendadas 11
5. Referencias 11

1. Introducción

La caminata aleatoria (random walk) es un modelo sencillo de difusión: una partícula parte del origen y, a cada paso, elige aleatoriamente uno de los movimientos permitidos. Aunque no pueda predecirse el punto final de una trayectoria particular, una cantidad grande de simulaciones permite observar regularidades estadísticas.

Este informe desarrolla y evalúa las variantes 1D y 2D, que constituyen el alcance exigido por la consigna. El equipo incorporó, además, una versión generalizada a D dimensiones y comprobó su funcionamiento en 3D y 4D. La implementación usa C y MPI y se estudia en cuatro entornos de cómputo.

1.1 Fundamento teórico

Se simulan N caminatas independientes de M pasos unitarios. En 1D, cada paso suma +1 o −1 con probabilidad 1/2. En 2D, se elige entre derecha, izquierda, arriba y abajo, cada dirección con probabilidad 1/4. La extensión a D dimensiones permite 2D direcciones, con modificación de una sola coordenada por paso.

Para cada caminata se obtiene la distancia euclidiana final R al origen. La magnitud estimada por Monte Carlo es $\bar{R} = (R_1 + R_2 + \cdots + R_N)/N$. Como los desplazamientos son independientes y de longitud unitaria, se cumple exactamente $E[R^2] = M$. Para M grande, la posición final se aproxima por una distribución normal en cada coordenada, de varianza M/D.

Caso	Distancia promedio teórica (M grande)
1D	$E[R] \approx \sqrt{(2M/\pi)}$
2D	$E[R] \approx \sqrt{(\pi M)} / 2$
D dimensiones	$E[R] \approx \sqrt{(2M/D)} \cdot \Gamma((D + 1)/2) / \Gamma(D/2)$

La función Γ es la función gamma. La fórmula general incluye los casos 1D y 2D. Las expresiones indican que E[R] crece proporcionalmente a √M; no debe confundirse E[R] con $\sqrt{E[R^2]} = \sqrt{M}$. Por ejemplo, para M = 1000 se espera E[R] ≈ 25,23 en 1D y ≈ 28,02 en 2D, frente a √M ≈ 31,62.

1.2 Objetivos

- Implementar en C y MPI las caminatas 1D y 2D, con parámetros configurables de pasos M, caminatas N y cantidad de procesos p.
- Contrastar la distancia promedio y la distribución final con las predicciones teóricas mediante simulación Monte Carlo.
- Medir speedup, eficiencia y tiempos de ejecución usando todas las cantidades consecutivas de procesos disponibles en cada equipo.
- Comparar los resultados entre cuatro sistemas y distinguir conclusiones observadas de hipótesis sobre el hardware.
- Como trabajo adicional, generalizar el programa a D dimensiones y comprobar los casos 3D y 4D.

2. Datos, implementación y metodología

El simulador genera las caminatas en memoria, sin datos de entrada externos. Cada ejecución informa la distancia promedio simulada y teórica, el error relativo y el tiempo de cómputo. La opción --sample permite guardar en CSV hasta 50.000 posiciones finales generadas por el proceso de rango 0, evitando escribir los millones de trayectorias simuladas durante los benchmarks.

2.1 Configuración de los experimentos

Tabla 1. *Sistemas utilizados en la campaña experimental.*

Equipo	Procesador	Recursos	Entorno	p evaluados
Clúster	AMD EPYC 7B12	Cuota equivalente a 12 CPU	Linux, contenedor	1–12
PC 6 núcleos	Intel Core i5-9400	6 núcleos / 6 hilos	WSL2, Windows	1–6
Netbook 4	AMD Ryzen 3 7320U	4 núcleos / 8 hilos	Fedora Linux	1–4
PC 2 núcleos	Intel Core i7-7500U	2 núcleos / 4 hilos	WSL2, Windows	1–4

> En el clúster, cpu.max = 1200000 100000 corresponde a una cuota de 12 CPU equivalentes; no demuestra la exclusividad de 12 núcleos físicos.

Se realizaron tres tipos de pruebas: validación del modelo (N = 1.000.000, M = 100, 500, 1000, 2000 y 5000); histogramas 1D y 2D (M = 1000 y muestra de hasta 50.000 posiciones); y escalabilidad fuerte (N = 10.000.000, M = 1000, con p desde 1 hasta el máximo de cada sistema). Para la extensión 3D y 4D también se ejecutaron las validaciones y los benchmarks.

El conjunto de escalabilidad incluyó 26 configuraciones de procesos por dimensión (12 + 6 + 4 + 4). Sumadas las cuatro dimensiones, se obtuvieron 104 configuraciones y 520 mediciones individuales, con cinco repeticiones por configuración.

2.2 Estrategia de paralelización

Se paralelizaron caminatas completas, no los pasos de una misma caminata. Esta elección evita dependencias entre procesos durante el cálculo, ya que cada trayectoria evoluciona a partir de su posición anterior pero es independiente de las demás.

Con N caminatas y p procesos, cada rango recibe $\lfloor N/p \rfloor$ caminatas y los primeros N mod p reciben una adicional. El desequilibrio máximo es una caminata. Los procesos solo sincronizan parámetros al inicio y reducen resultados al final, por lo que el problema es prácticamente embarazosamente paralelo.

2.3 Implementación MPI y generación aleatoria

La posición de cada caminata se mantiene en un vector de D enteros. En cada paso se sortea una de 2D direcciones y se suma ±1 al eje correspondiente. Al finalizar se acumula la distancia $R = \sqrt{(x_1^2 + \cdots + x_D^2)}$; no se almacenan trayectorias completas. El trabajo dominante consiste en N·M actualizaciones, aunque el cálculo final de la norma tiene también un costo proporcional a D por caminata.

El generador pseudoaleatorio es PCG32. Se selecciona cada dirección mediante multiplicación y rechazo, para evitar el sesgo que podría introducir una reducción modular directa. Se utiliza una semilla base fija y cada rango MPI dispone de un stream diferente:

```
pcg32_srandom_r(&rng, base_seed, base_stream + (uint64_t)rank);
```

Durante el desarrollo se consideraron tres estrategias: semillas separadas linealmente ($S_r = S_0 + r \cdot B$), streams diferentes de PCG32 y leapfrog, que intercala valores entre procesos mediante saltos sobre una secuencia. Se probó primero la técnica de semillas lineales, pero apareció un problema de desbordamiento descrito por el equipo como «desborde del búfer». No se conservan el código ni la traza de ese fallo para determinar su causa. Se optó por los streams de PCG32, que evitan comunicación y asignan secuencias diferentes por rango. Esto no constituye una demostración formal de independencia estadística.

Con parámetros y cantidad de procesos iguales se reproducen las mismas secuencias. Si cambia p, cambia el reparto y también las trayectorias individuales. Las cinco ejecuciones por configuración emplearon idénticas semillas: estiman la variabilidad temporal, pero no equivalen a cinco muestras estadísticas independientes.

Las principales operaciones MPI fueron MPI_Bcast (parámetros), MPI_Barrier (sincronización previa al cronómetro) y MPI_Reduce, con MPI_SUM para la suma de distancias y MPI_MAX para seleccionar el tiempo local máximo. El proceso 0 informa el resultado final.

2.4 Medición y reproducibilidad

Cada proceso toma el tiempo con MPI_Wtime después de la barrera y se considera el mayor tiempo local para representar la duración paralela. La medición llega hasta la reducción global de distancias y excluye el arranque de mpirun; tampoco incluye la difusión de parámetros si esta ocurre antes de la barrera inicial. Se publican las medianas de cinco repeticiones.

Se utilizan las métricas S(p) = T(1)/T(p) (speedup) y E(p) = S(p)/p (eficiencia). La referencia T(1) es siempre la ejecución del propio binario MPI con p = 1 en la misma máquina. Las versiones secuenciales compiladas con -O0 no se usan de referencia porque las variantes MPI se compilaron con -O3 -march=native.

Siguiendo el modelo de la cátedra, el tiempo global puede descomponerse en cálculo, comunicación y espera. Para esta aplicación se espera que predomine el cálculo; no obstante, no se registraron por separado esos componentes, por lo que la identificación de causas de pérdida de eficiencia requiere nuevas mediciones.

El proyecto automatiza la campaña con scripts/benchmark.sh, el Makefile y scripts/generar_graficos.py (pandas, NumPy y Matplotlib). Se comprobó que las medianas recalculadas desde los registros crudos coinciden con los CSV resumidos en las 104 configuraciones. Un ejemplo de ejecución en el clúster es:

```
PROCS="1 ... 12" RUNS=5 MPI_FLAGS="--bind-to core --map-by core" make benchmark DIMS="3 4"
```

3. Resultados

3.1 Validación de 1D y 2D

La Tabla 2 presenta los resultados de las dos dimensiones centrales de la consigna, obtenidos en el clúster para N = 1.000.000 caminatas y M = 1000 pasos. En ambos casos la distancia promedio simulada coincide estrechamente con el valor aproximado teórico.

Tabla 2. *Distancia promedio de 1D y 2D ($N = 10^6$; $M = 1000$).*

Caso	Simulada	Teórica	Error relativo
1D	25,2356	25,2313	0,017 %
2D	28,0305	28,0250	0,020 %

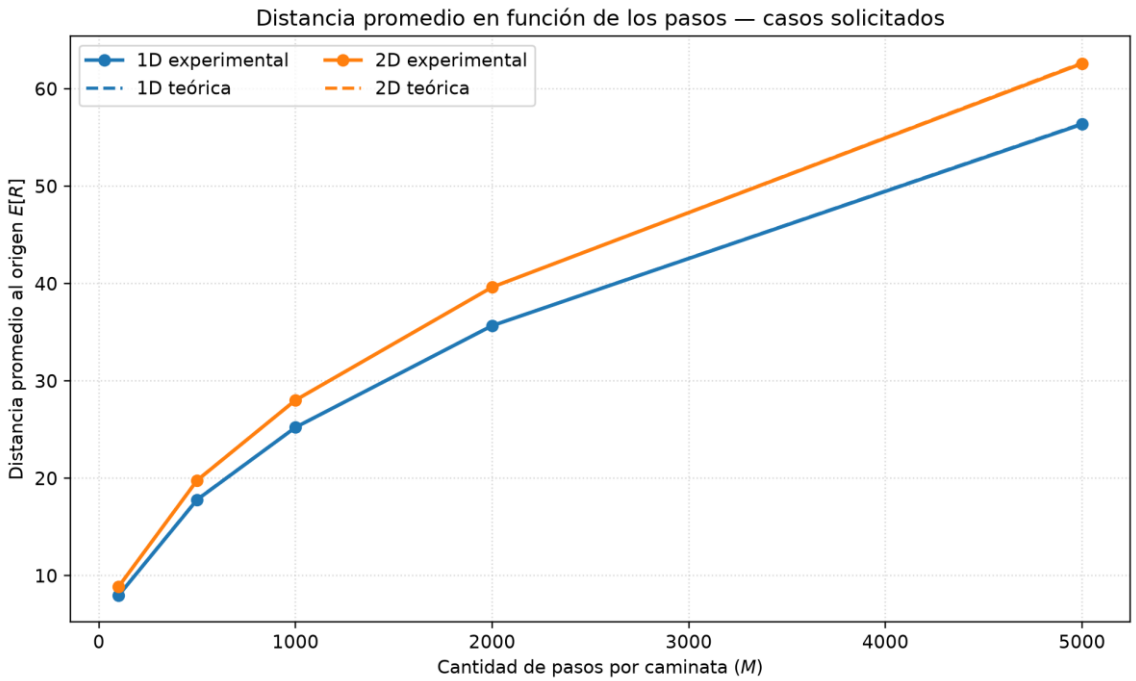


Figura 1. *Distancia promedio frente a los pasos: resultados de 1D y 2D y curvas teóricas.*

En el rango M = 100 a 5000, las trayectorias medias siguen el crecimiento proporcional a √M. El mayor error relativo observado entre las 20 combinaciones validadas (incluidas las extensiones) fue 0,157 %. Este valor es pequeño frente a las distancias medidas, pero no implica exactitud determinista: las realizaciones de Monte Carlo presentan fluctuaciones estadísticas y las fórmulas son asintóticas para M grande.

3.2 Distribución de posiciones finales

En 1D, el histograma de posiciones finales para M = 1000 es aproximadamente simétrico alrededor de cero y sigue la forma normal predicha para un paseo sin deriva; su desviación típica teórica es √1000 ≈ 31,6.

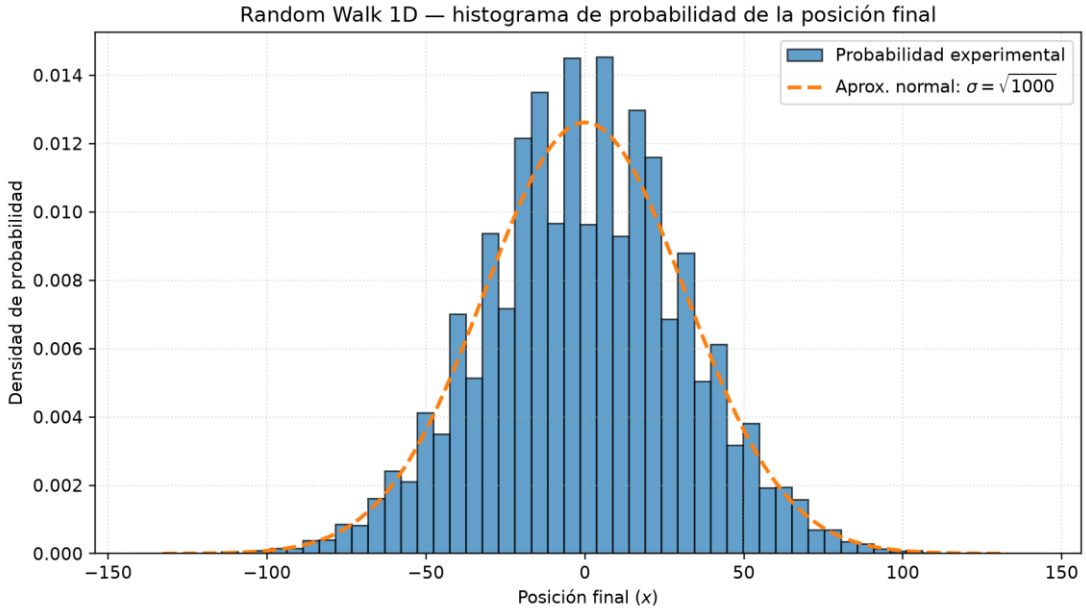


Figura 2. *Histograma 1D de posición final y aproximación normal para M = 1000.*

La alternancia de barras altas y bajas no indica un defecto del simulador. Como M = 1000 es par, solo pueden alcanzarse posiciones pares. Los intervalos del histograma tienen ancho cercano a cinco unidades y contienen alternadamente dos o tres valores permitidos.

En 2D, la densidad de posiciones muestra simetría aproximadamente circular, sin privilegio de un eje, como se espera del movimiento aleatorio sobre la retícula cuadrada. La circunferencia de referencia tiene radio E[R] ≈ 28,02.

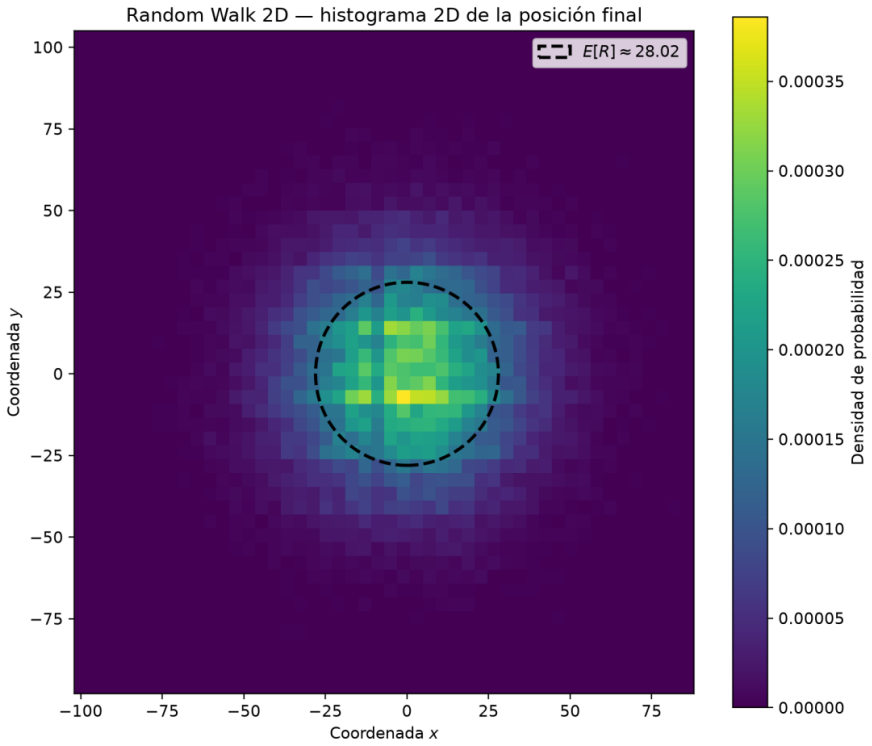


Figura 3. *Distribución bidimensional de posiciones finales y circunferencia de radio E[R], M = 1000.*

3.3 Escalabilidad en el clúster

Con N = 10.000.000 y M = 1000 fijos (strong scaling), tanto 1D como 2D se aceleran casi linealmente hasta p = 12. La Tabla 3 muestra las cifras principales del proyecto.

Tabla 3. *Resultados principales del clúster con 12 procesos.*

Caso	T(1) [s]	T(12) [s]	S(12)	E(12)
1D	14,351	1,198	11,978	99,82 %
2D	65,785	5,484	11,996	99,97 %

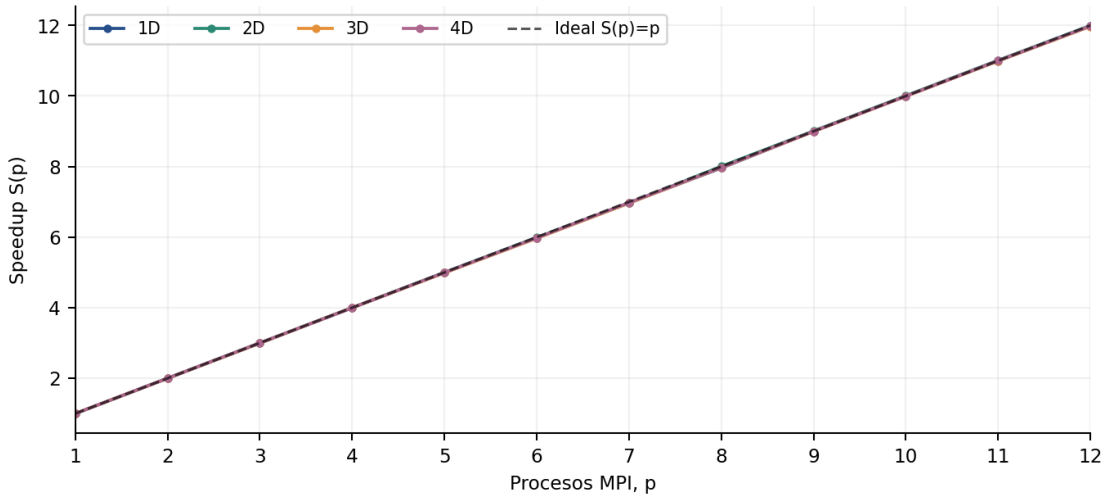


Figura 4. *Speedup en el clúster. Se muestran 1D y 2D junto con las curvas casi superpuestas de las extensiones 3D y 4D.*

El trabajo independiente entre procesos y el reducido volumen de datos comunicados son compatibles con este comportamiento casi ideal. Sin embargo, no se midieron por separado los tiempos de comunicación y no corresponde extrapolar los resultados a más de 12 procesos ni a un clúster multinodo.

3.4 Comparación entre los cuatro sistemas

La comparación entre máquinas requiere distinguir eficiencia y velocidad absoluta. En la Tabla 4 se informa la aceleración con el mayor número de procesos que se probó en cada entorno; los máximos de p no son equivalentes entre equipos.

Tabla 4. *Speedup de 1D y 2D con p máximo de cada sistema.*

Sistema (p máximo)	1D: S(p)	2D: S(p)	1D: E(p)	2D: E(p)
Clúster (12)	11,978	11,996	99,82 %	99,97 %
PC 6 núcleos (6)	5,162	5,714	86,03 %	95,23 %
Netbook 4 (4)	3,674	3,646	91,84 %	91,16 %
PC 2 núcleos / SMT (4)	2,257	2,821	56,43 %	70,53 %

Para una comparación a igual cantidad de procesos, se considera p = 4, presente en los cuatro sistemas. El clúster se mantiene cerca del 100 %; la PC de seis núcleos ronda el 97 %; la netbook alcanza cerca del 91 %; y la PC con dos núcleos físicos pierde eficiencia nominal al asignar cuatro procesos a cuatro hilos lógicos, que no equivalen a cuatro núcleos independientes.

Tabla 5. *Eficiencia con p = 4 en los casos obligatorios.*

Sistema	1D	2D
Clúster	99,96 %	100,01 %
PC 6 núcleos	97,01 %	97,60 %
Netbook 4	91,84 %	91,16 %
PC 2 núcleos / SMT	56,43 %	70,53 %

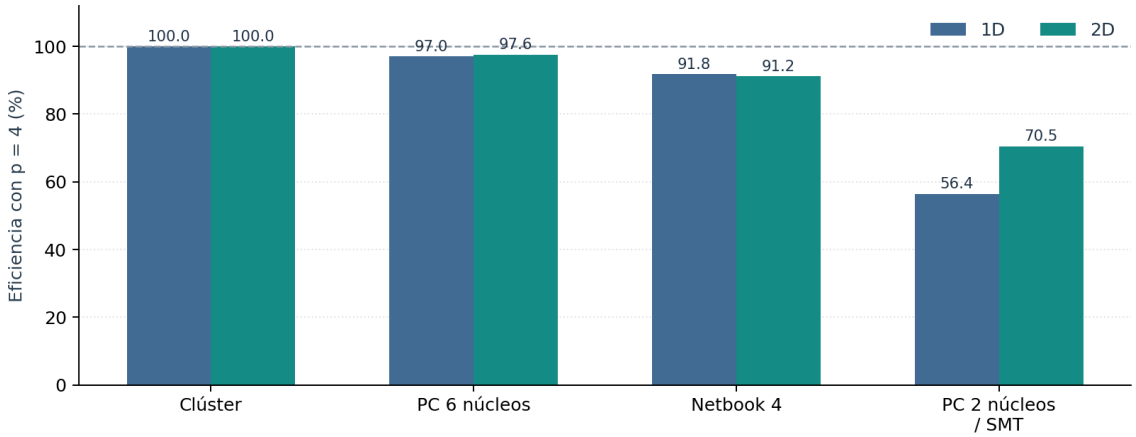


Figura 5. *Comparación de eficiencia con cuatro procesos: únicamente los casos 1D y 2D.*

Las diferencias son compatibles con efectos de frecuencia, ocupación del sistema operativo, límites de recursos y topología de núcleos e hilos. No se controlaron ni midieron todas estas variables, por lo que deben tratarse como hipótesis de interpretación, no como causas demostradas.

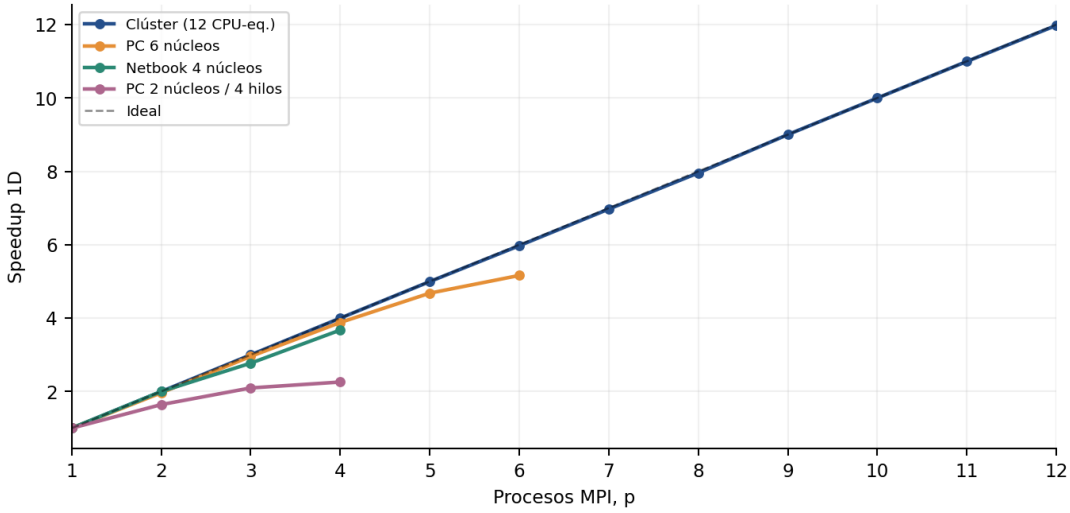


Figura 6. Evolución del speedup 1D para cada sistema; cada serie termina en su p máximo.

3.5 Tiempo absoluto y anomalía de 2D

Una eficiencia más alta no garantiza un menor tiempo de ejecución: el speedup compara un sistema consigo mismo. La Tabla 6 presenta los tiempos con p = 4 de las dos variantes requeridas.

Tabla 6. Tiempos medianos absolutos ($N = 10^7$; $M = 1000$).

Sistema	T(1) 1D [s]	T(4) 1D [s]	T(4) 2D [s]
Clúster	14,351	3,589	16,445
PC 6 núcleos	13,348	3,440	15,382
Netbook 4	11,378	3,097	14,210
PC 2 núcleos / SMT	15,289	6,773	24,362

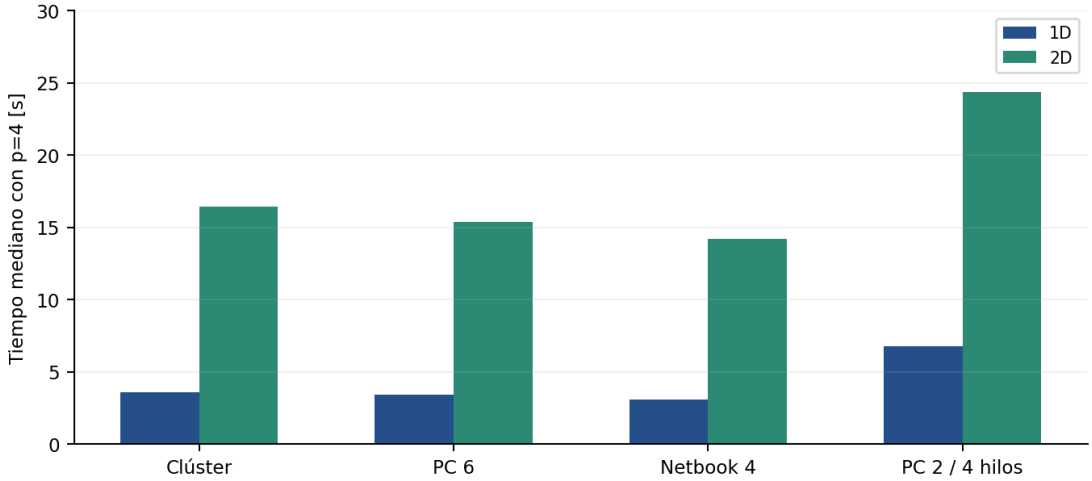


Figura 7. Tiempos medianos para p = 4: 1D frente a 2D en cada sistema.

La netbook resuelve 1D más rápido que el clúster con p = 4 (3,097 s frente a 3,589 s), aunque su eficiencia es inferior. El clúster resulta más rápido al poder utilizar hasta 12 procesos dentro de su cuota (1,198 s en 1D). Por ello, la escalabilidad no equivale a una clasificación absoluta de CPU.

La implementación 2D destaca por un costo mayor: en el clúster, con un proceso, demora 65,785 s frente a 14,351 s en 1D. A igual $N \cdot M = 10^{10}$ pasos, equivale a 6,58 ns por paso en 2D y 1,44 ns en 1D. Como referencia interna, las ejecuciones de la versión generalizada en 3D y 4D rondan 1,78 ns por paso, es decir que 2D tarda alrededor de 3,7 veces más que 3D pese a tener menos dimensiones.

Una explicación posible, todavía no verificada, es que la selección especializada de la dirección en random_walk_2d.c introduce saltos condicionales costosos frente al procedimiento usado en random_walk_nd.c. Para verificarla habría que medir random_walk_nd con D = 2 en el mismo entorno y realizar perfiles de CPU. Esta diferencia afecta el tiempo absoluto, aunque las métricas de speedup de cada binario sigan siendo válidas.

3.6 PC de dos núcleos: hilos lógicos

El i7-7500U posee dos núcleos físicos y cuatro hilos lógicos (SMT/Hyper-Threading). Las ejecuciones con p = 4 utilizaron --oversubscribe --bind-to none. Cuatro procesos superan el número de núcleos físicos, pero no el número de CPU lógicas; no se estudió p > 4 ni se fijó la afinidad.

Tabla 7. *Aceleración y eficiencia nominal en la PC con SMT.*

Caso	S(4)	E(4) = S(4)/4
1D	2,257	56,43 %
2D	2,821	70,53 %

La eficiencia E(4) compara contra cuatro unidades ideales de cómputo y, por tanto, no equivale a la utilización física de cuatro núcleos. El cociente S(4)/2 puede servir como referencia descriptiva frente a dos núcleos ideales, pero no mide directamente la ganancia de SMT. Para estimar la mejora observada al pasar de dos a cuatro procesos se necesita T(2)/T(4) = S(4)/S(2), y para atribuirla específicamente a SMT se requieren afinidad y condiciones de frecuencia controladas.

El mejor speedup observado en 2D respecto de 1D podría estar relacionado con la diferente mezcla de instrucciones de ambas implementaciones. No se midieron fallos de predicción, uso de unidades de ejecución ni frecuencia, por lo que no se afirma una causa específica.

3.7 Extensión a 3D y 4D

Como mejora no requerida por la consigna, se implementó un simulador generalizado a D dimensiones y se probaron 3D y 4D. La extensión conserva la estrategia de reparto por caminatas y las operaciones colectivas MPI; únicamente cambia el número de coordenadas y direcciones posibles.

Tabla 8. *Síntesis de los resultados adicionales de 3D y 4D.*

Caso	R̄ simulada	E[R] teórica	Error (M = 1000)	S(12) clúster	E(12)
3D	29,1369	29,1346	0,008 %	11,972	99,77 %
4D	29,7570	29,7250	0,108 %	11,983	99,86 %

Estos resultados documentan que la generalización se ejecutó y conservó una buena escalabilidad para el tamaño de problema ensayado. No se realiza aquí un estudio estadístico detallado de 3D y 4D: el foco de validación, las comparaciones gráficas y la discusión principal corresponden a 1D y 2D. Los errores observados en una muestra de Monte Carlo no sustituyen un análisis entre múltiples semillas independientes.

4. Conclusiones

Se implementó y validó un simulador MPI de Random Walk 1D y 2D, de acuerdo con el alcance principal del proyecto. Las distancias promedio concuerdan con la teoría, crecen aproximadamente como $\sqrt{M}$ y los histogramas muestran las distribuciones esperadas: una campana centrada en cero en 1D y simetría aproximadamente circular en 2D.

La estrategia de distribuir caminatas completas resultó adecuada. En el clúster se obtuvo un speedup de 11,978 (1D) y 11,996 (2D) con 12 procesos, ambos próximos al ideal. En las computadoras personales, los resultados fueron más variables. Las mediciones respaldan que la aplicación tiene poca necesidad de comunicación, pero no permiten separar de forma causal los efectos de hardware, planificación y frecuencia.

La evaluación también mostró que speedup y tiempo absoluto responden preguntas distintas. Además, la versión especializada de 2D resultó considerablemente más lenta por paso que la generalizada usada como referencia. Ese

hallazgo señala una mejora concreta que debe evaluarse mediante comparación de binarios y perfiles, no mediante suposiciones sobre la microarquitectura.

Finalmente, la generalización a 3D y 4D constituye una contribución adicional del equipo: muestra la reutilización de la arquitectura MPI y permite explorar nuevas dimensiones sin cambiar el esquema básico de distribución del trabajo.

4.1 Limitaciones

- La escalabilidad fuerte se midió con un único tamaño (N = 10.000.000, M = 1000). No se realizó weak scaling ni un barrido de tamaños menores.
- El clúster fue un contenedor con cuota de 12 CPU equivalentes en un solo nodo; no se evaluó comunicación entre nodos ni escalamiento por encima de doce procesos.
- Las cinco repeticiones por configuración usan semillas fijas: reflejan variación temporal y no incertidumbre estadística entre realizaciones independientes.
- No se registraron tiempos locales por proceso ni costos desglosados de cálculo, comunicación y espera. Tampoco se monitorizaron frecuencia de CPU o fallos de predicción.
- Los entornos son heterogéneos (Fedora, WSL2 y Linux en contenedor), por lo que los tiempos absolutos no son una comparación controlada de procesadores.
- No se fijó la afinidad de la PC de dos núcleos ni se ensayó p > 4; la independencia estadística entre streams de PCG32 no fue demostrada formalmente.
- El código del clúster y de la netbook fue contrastado como idéntico; de las otras dos computadoras se conservaron los resultados, pero no el código fuente completo empleado.

4.2 Mejoras recomendadas

Se priorizan cuatro líneas de mejora que amplían directamente la validez de los resultados y la calidad de la implementación:

1. Explorar más tamaños de problema. Variar N y M para determinar cuándo deja de convenir aumentar p; agregar pruebas de weak scaling, manteniendo el trabajo por proceso aproximadamente constante.

2. Perfilar y optimizar la implementación 2D. Registrar tiempos de cálculo, comunicación y espera por proceso, además de frecuencia y eventos relevantes de CPU; comparar la versión 2D especializada con random_walk_nd en D = 2 antes de cambiar el algoritmo.

3. Fortalecer la validación estadística. Repetir las pruebas principales con varias semillas base independientes, reportar media y error estándar, y documentar el esquema de inicialización de streams del generador.

4. Evaluar una implementación híbrida MPI + OpenMP. Comparar, dentro de la cuota de doce CPU equivalentes, combinaciones de procesos e hilos tales como 12×1, 6×2, 4×3 y 3×4, registrando afinidad y rendimiento.

5. Referencias

[1] Material de la cátedra: Computación de Alto Rendimiento para aplicaciones numéricas. Métodos Numéricos II, FACET-UNT (2026).

[2] MPI Forum. MPI: A Message-Passing Interface Standard. https://www.mpi-forum.org/docs/

[3] Open MPI Project. Open MPI Documentation. https://docs.open-mpi.org/

[4] O'Neill, M. E. (2014). PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation. https://www.pcg-random.org/paper.html

[5] O'Neill, M. E. Using the Full C Implementation – PCG. https://www.pcg-random.org/using-pcg-c.html

[6] Free Software Foundation. GNU Make Manual. https://www.gnu.org/software/make/manual/make.html

[7] Repositorio del proyecto (código, scripts y resultados). https://github.com/JavierRodriguez3/Random-walk-Grupo5